PORTER: A Plan-Committed Router for Multi-Core Distributed Quantum Computers

Fuchao Sheng, Yongjian Xie, Yongming Li, Sanjiang Li *

***Abstract*—A distributed quantum computer runs one circuit across several small cores joined by links that distribute entanglement on demand, and the compiler's figure of merit is how few Einstein–Podolsky–Rosen (EPR) pairs it spends on gates that straddle cores. We present PORTER, a compiler for such machines. It unifies the three known inter-core primitives — state teleportation, gate teleportation and multi-gate cat-entangler packets — under one *plan-committed* decision loop that chooses a complete inter-core action before generating the SWAPs that make it executable; adds a two-tier capacity-relief mechanism that routes qubits toward reservoir cores when every candidate is blocked; and builds its initial layout with a deterministic, seed-free *temporal locality-greedy* heuristic in place of a seeded search. On MQT-Bench suites at 25, 64 and 80 logical qubits a single PORTER pass lowers geometric-mean EPR consumption against published TELESABRE by 59.6%, 41.6% and 68.2%, and completes all twenty-four instances including seven on which TELESABRE does not converge. Against DSABRE, a concurrent compiler revised during this work and therefore re-run rather than quoted, PORTER is competitive: $1.07\times$ its pooled EPR at $0.58\times$ its compile time under each compiler's own protocol, *ahead* of it on forty-one identical externally supplied layouts, on both heavy-hex core graphs, and on a 100–360-qubit scale-up suite, where selecting the lookahead horizon per circuit turns a $1.45\times$ deficit into a $0.72\times$ lead. A controlled ablation then isolates what each design decision buys, and two of them do not buy EPR pairs — which we report as measured. Every run is certified by an independent replay verifier; the implementation, verifier and harness are public.**

***Index Terms*—distributed quantum computing, quantum circuit compilation, multi-core quantum architectures, gate teleportation, cat-entangler packets, plan-committed routing, entanglement cost**

I. INTRODUCTION

A distributed quantum computer (DQC) runs one circuit across several small quantum processing units — *cores* — joined by links that distribute entanglement on demand [1], [2]. What such a machine is short of is not gates but entanglement. Any two-qubit gate whose operands sit in different cores has to be bought with Einstein–Podolsky–Rosen (EPR) pairs, and on current hardware distributing a pair is far slower and far less reliable than any operation performed inside a core [3], [4]. How few pairs a compiler spends is therefore the figure of merit that matters.

There is more than one way to spend a pair. A *state* can be teleported (TELEDATA), relocating a logical qubit so every later gate it meets in its new core is free [5]. A *gate* can be teleported (TELEGATE) through a cat-entangler, executing one remote CX with neither operand moving [6], [7]. Or one cat-entangled resource can be held open across a run of remote CX gates sharing a control, so the run costs one pair instead of one each — a *packet*, in the spirit of AutoComm's gate-sequence amortisation [8]. They differ in exactly one respect that matters: how many gates a pair is spread over, which is a property of the circuit structure where the primitive is applied and not of the primitive itself, so none of the three dominates.

The choice must be made before its price is known. Each alternative becomes executable only after its own preparation — the right operand beside the right communication port, that port free, the destination holding a spare slot — and preparation is work the compiler will not perform unless it selects that alternative. Scoring an action together with the preparation it currently needs therefore ranks candidates by the layout the compiler happens to be sitting in rather than by the progress they buy, and staging a losing candidate still perturbs the layout, so effort is spent preparing actions never taken.

PORTER — a *Plan-Oriented Router for Teleportation-Enabled aRchitectures* — keeps the two questions apart. Its unit of choice is a *plan*: a complete, committable specification of one inter-core action — which primitive fires, which gate or same-control run it resolves, in which direction, and hence which readiness conditions it needs — with the SWAP sequence that would establish those conditions deliberately left out. Whole plans are compared on the gate progress they buy; one is committed; only then are preparation SWAPs generated, for that plan alone. Around this loop PORTER adds a two-tier capacity-relief mechanism that manufactures free slots when every candidate is capacity-blocked, and a deterministic, seed-free initial layout replacing the seeded best-of-N search its predecessors run.

Results. We evaluate on MQT-Bench [9] suites at 25, 64 and 80 logical qubits, on a 100–360-qubit scale-up series, and on two networks of IBM heavy-hex cores — forty-eight circuit–architecture instances. Against TELESABRE [10], the published state of the art for this cost model, PORTER is decisively better: a single pass lowers geometric-mean EPR consumption by 59.6%, 41.6% and 68.2% on the three suites, and it completes every instance, including the seven — among them an 80-qubit, 23,381-CX circuit — on which TELESABRE does not converge in three seeds.

The second comparison is less one-sided and we report it in full. DSABRE [11] is concurrent work on the same problem whose router was substantially revised while this

F. Sheng, Y. Xing, and Y. Li are with the School of Mathematics and Statistics, Shaanxi Normal University, Xi'an, China; S. Li is with the Centre for Quantum Software and Information, University of Technology Sydney, Australia (e-mail: sanjiang.li@uts.edu.au).

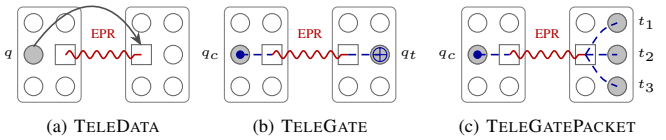
*ORCID: 0000-0002-3332-2546.

paper was in preparation, so we re-ran it from source rather than quoting its tables. Against that version PORTER is *competitive rather than dominant*. Under each compiler's own protocol PORTER costs $1.07\times$ DSABRE's pooled geometric-mean EPR at $0.58\times$ its compile time; restoring a comparable pass budget reaches $1.05\times$ at $0.74\times$. Routed from identical externally supplied layouts at matched pass budgets PORTER moves *ahead* across forty-one instances (126.6 against 134.2), and on both heavy-hex core graphs it takes the suite. On the 100–360-qubit series it now *leads* by $0.72\times$ — a reversal traced in Section IV-H to a single mistuned constant, the lookahead horizon, which no one value serves across the benchmark. The two compilers share four load-bearing design decisions, set out in Section V so that neither is credited with the other's.

What the mechanisms are worth. Because the two compilers are close relatives, a head-to-head number says little about which design decision earns its place, so we ablate each of PORTER's in isolation and report the outcome as measured rather than as advertised. Two of the four answers are negative. Capacity relief is what makes a deterministic-greedy design complete at all — without it four of six 80-qubit instances deadlock — and the initial layout is what separates the two compilers below roughly a hundred physical qubits. But plan commitment reaches the same EPR as re-scoring the whole front every iteration, within 0.6% and on the same number of routing moves, so what it buys is 1.3–$3\times$ less compile time rather than better routes. And the packet primitive, which exists to amortise a pair across a run of gates, is *outperformed at 80 qubits by removing gate teleportation altogether*, by 32.8%.

Contributions. We contribute PORTER itself — a deterministic, seed-free DQC compiler built around the plan-first remote stage of Section III-E, the two-tier capacity relief of Section III-H, and the temporal locality-greedy layout of Section III-A, with an optional refinement schedule that trades compile time back for EPR pairs. We contribute the observation that the teleport-lookahead horizon, a parameter every SABRE-descended router fixes by constant, has no good constant: the best value spans a factor of thirty-two across our benchmark, is predicted in direction but not in magnitude by how dense the circuit's interaction graph is, and is cheaply chosen per circuit by routing a small pruned candidate set and keeping the best (Section IV-H). Doing so is what carries PORTER from $1.45\times$ behind DSABRE to $0.72\times$ ahead on the scale-up series, and removes its only unrouted instance. We contribute an evaluation across forty-eight circuit–architecture instances — under each compiler's own protocol, under shared fixed layouts, with each compiler routing the other's layout, and on two heavy-hex core graphs — in which PORTER beats published TELESABRE decisively and is competitive with concurrent DSABRE, and every reported run is certified by an independent replay verifier that re-derives gate coverage, dependency order and physical legality from the emitted operation stream alone. And we contribute an ablation that decides what to believe about each mechanism (Section IV-I), including the two answers that are negative.

The PORTER implementation, the replay verifier, the full cross-router comparison harness, and the raw per-circuit results are publicly available at https://github.com/ebony72/porter.



(a) TELEDATA (b) TELEGATE (c) TELEGATEPACKET

Fig. 1. The three communication primitives; each consumes one EPR pair between the two communication ports (squares). (a) TELEDATA moves qubit q (which must sit adjacent to its free port) onto the free destination port. (b) TELEGATE executes one remote CX through a cat-entangler [7] (Appendix E): the target-side port acts as a proxy for the control q_c; neither qubit moves. (c) TELEGATEPACKET brackets an ordered run of remote CX gates sharing the control q_c between one cat-entangler/disentangler pair, amortising a single EPR pair over the whole run; each target is routed next to the port in turn while the ports stay reserved.

II. BACKGROUND

A. Quantum Circuits and Routing

A circuit acts on n logical qubits through a sequence of gates; we work in the restricted basis $\{\texttt{id}, \texttt{rz}, \texttt{sx}, \texttt{x}, \texttt{cx}\}$ of MQT-Bench's Qiskit-opt3-transpiled circuits [9], i.e. arbitrary one-qubit gates plus the two-qubit CX. A circuit is a directed acyclic graph (DAG): for every logical qubit, consecutive gates acting on it are joined by a dependency edge. The *front* is the set of gates with no unexecuted predecessor; only front gates can execute or be routed towards execution. Gates sharing a qubit — two CX gates with the same control, say — are chained by a dependency edge and therefore *never* appear in the front together, a fact that matters for packet collection (Section III-G).

Physical qubits are partitioned into *cores*; each core has an intra-core coupling graph, and pairs of cores may be joined by an inter-core link whose two endpoints are designated *communication ports*. A *layout* maps every logical qubit to a physical position. A two-qubit gate is *local* if both operands share a core and *remote* otherwise; routing a local gate that is not yet adjacent requires intra-core SWAPs, and routing a remote gate requires an inter-core communication primitive. We write $\kappa_\ell(q)$ for the core holding q under layout ℓ, $d_{\mathrm{L}}(c, p, p')$ for intra-core distance and $d_{\mathrm{C}}(c, c')$ for core-graph distance.

B. Communication Primitives

Three inter-core primitives are in play, sketched in Figure 1; each consumes exactly one EPR pair.

State teleportation (TELEDATA). Consumes one EPR pair to move a logical qubit from a communication port in one core to a free port in another, after which every later gate that qubit meets in its new core is local. Its value is therefore *latent*: it is paid once and repaid over however many future gates the relocation makes local.

Gate teleportation (TELEGATE). Executes a single remote CX with neither operand moving. One EPR pair entangles the two communication ports; a *cat-entangler* — $\mathrm{CX}(q_c, e_A)$, measurement of e_A, and a classically controlled X on e_B

— leaves (q_c, e_B) in the cat state $\alpha|00\rangle + \beta|11\rangle$, so every CX controlled by the remote port e_B acts as if controlled by q_c. A *cat-disentangler* — Hadamard on e_B, measurement, a classically controlled Z on q_c — restores q_c and frees both ports (Appendix E). Its value is entirely *direct*: exactly one gate, and nothing is left behind.

Same-control packets (TELEGATEPACKET). When several remote CX gates share the same control and their targets lie in one core, the run can be bracketed between a *single* cat-entangler and disentangler: the ports stay reserved, each target is routed beside e_B in turn, and the whole run costs one EPR pair instead of one each. This is the amortisation idea of AutoComm [8], specialised to the shared-control case and applied inside a routing loop rather than as a pre-pass. Because same-control gates are chained by DAG dependency edges they never appear in the front together, so a packet cannot be read off the front and has to be collected by forward simulation (Section III-G).

The three differ in exactly one respect that matters for scoring: how many gates one pair is spread over. TELEDATA amortises over an unknown future, TELEGATE over exactly one gate, and TELEGATEPACKET over a run whose length the compiler must discover. Section III-F makes that difference explicit as a direct/latent/lookahead decomposition, and Section IV-I measures whether the richer vocabulary is worth having.

C. SABRE

SABRE [12] routes a single-core circuit by repeatedly selecting the SWAP that minimises a front-layer distance term plus a weighted extended-set lookahead term, both expressed as Δ (old distance minus new distance) on gates sharing a qubit with the candidate SWAP. PORTER inherits this scoring form for its intra-core stage (Section III-D) and extends the same direct/lookahead decomposition to its inter-core plan scorer (Section III-F).

III. THE PORTER COMPILER

Given a circuit and an initial layout ℓ_0 (which PORTER builds itself, Section III-A), PORTER repeats the following loop (Algorithm 1) until every gate has been emitted or a configurable iteration budget is exhausted. Algorithm 1 charts one iteration; its shaded region is the plan-first remote stage that distinguishes PORTER from compilers deciding at single-action granularity; Appendix F walks through it on a concrete front where it changes the outcome.

Algorithm 1 PORTER `compile`(C, ℓ_0)

$\ell \leftarrow \ell_0$; initialise DAG active/frontier sets from C
while DAG has active gates **do**
  $F \leftarrow$ DAG front (bounded by `dag_front_limit`)
  classify F into singles, direct-local CX, local-CX-needing-SWAP, adjacent-remote CX, non-adjacent-remote CX *(Sec. III-C)*
  if singles or direct-local CX present **then**
    execute them all; remove from DAG; **continue**
  end if
  if local-CX-needing-SWAP present **then**
    generate candidate intra-core SWAPs, apply the best-scoring one (Eq. 1); **continue** *(Sec. III-D)*
  end if
  generate normal remote plans $\mathcal{P}$ and blocked-teleport plans $\mathcal{B}$ *(Sec. III-F)*
  if $\mathcal{P} \neq \emptyset$ **then**
    $p^* \leftarrow$ best ready-or-preparable plan in $\mathcal{P}$
    if some $b \in \mathcal{B}$ has gain $\gg$ gain(p^*) **then**
      $p^* \leftarrow$ best opportunistic capacity-relief plan for b *(Sec. III-H)*
    end if
  else
    BFS from each blocked target core to its nearest reservoir; $p^* \leftarrow$ best tail-relief plan *(Sec. III-H)*
  end if
  if p^* is `None` **then**
    report deadlock; abort
  end if
  apply p^*'s own preparation SWAPs until ready, then execute it
end while
return emitted operations, final layout, stats

A. Initial Layout

PORTER builds its starting layout ℓ_0 with one deterministic, seed-free heuristic — no external partitioner, no best-of-N search — and respects the capacity invariant the routing loop depends on: every core keeps at least $f_{\min}$=2 free positions, so from the first iteration any core can serve as a TELEDATA destination and as a relief reservoir.

Temporal locality-greedy. Three deterministic steps. *(i) Weighting*: each CX gate $g = (q_i, q_j)$ contributes $\gamma^{\ell(g)}$ to the interaction weight $w(q_i, q_j)$, where $\ell(g)$ is g's as-soon-as-possible layer in the CX-only DAG and γ=0.99, so early interactions weigh more than late ones while repeated interactions accumulate. *(ii) Core assignment*: logical qubits are visited in the order they first appear as a CX operand and each is placed in the core holding its heaviest already-placed neighbour when that core has headroom above $f_{\min}$, falling back to the core with the most remaining reserved capacity. *(iii) Physical placement*: within each core, logicals are laid onto positions by descending interaction weight onto ascending intra-core centrality, so the most-connected qubits take the most central slots.

The steps are complementary. The temporal discount front-loads the opening layers, co-locating the pairs that must interact soonest; accumulated multiplicity plus first-appearance order tracks chain-structured circuits, where only one disjoint pair is exposed per layer and a balanced-first-layer rule would scatter the chain across cores; and the centrality step keeps hub qubits cheap to reach. The construction costs microseconds and consumes no randomness, so PORTER has no seed to report.

B. Backward Refinement

PORTER's default protocol is one forward pass from ℓ_0. We also report an optional refinement schedule, because Section IV-E shows a fixed layout is not the whole difference between the two compilers: given the *same* externally supplied layout, DSABRE's forward–backward–forward schedule extracts more from it than a single forward pass does.

The schedule is SABRE's bidirectional layout refinement [12] applied to whole routes. Let $R(C, \ell)$ denote routing C from ℓ and ℓ' the layout it ends on. Pass 1 computes $R(C, \ell_0)$; pass 2 computes $R(C^{\mathrm{rev}}, \ell'_1)$ on the reversed circuit and *discards its route*, keeping only ℓ'_2; pass 3 computes $R(C, \ell'_2)$. Reversing C reverses every dependency edge, so routing it from the forward pass's end state yields a layout adapted to the *start* of the forward circuit. Odd pass counts above three repeat the cycle; P=5 throughout. We report the best forward pass by EPR, so refinement can never report worse than the single pass. It adds no randomness and its whole cost is routing passes — P calls where the default makes one, against DSABRE's nine.

It recovers less here than it does for DSABRE, and the reason is instructive. Refinement consumes a post-routing layout, and routing destroys the structure ℓ_0 was built with: on 64-qubit AE ℓ_0 leaves $(2, 2, 2, 2, 8, 16)$ free slots per core — packed cores plus two reservoirs, exactly the shape the capacity machinery wants — and one forward pass scrambles that to $(14, 1, 12, 2, 1, 2)$. Intermediate passes therefore often regress (64q AE 177 $\rightarrow$ 308, 64q QFT 262 $\rightarrow$ 326), which only the best-pass rule keeps harmless. Where it pays it pays well: 64q QNN 864 $\rightarrow$ 537, 64q Multiplier 1,358 $\rightarrow$ 763.

C. Front Classification

Every front gate is classified under the current layout ℓ. 1Q gates are always directly executable; a CX $g = (q_i, q_j)$ with both operands in core c is *direct-local* if $d_{\mathrm{L}}(c, \ell(q_i), \ell(q_j)) = 1$ and *local-needing-SWAP* otherwise; a CX whose operands lie in different cores is *adjacent-remote* if those cores are core-graph neighbours and *non-adjacent-remote* otherwise. Directly executable gates drain first and completely, intra-core SWAPs are exhausted next, and remote-stage decisions are taken only once no front gate is intra-core. The front is size-capped at 96 gates to bound per-iteration cost, a cap that never binds here since front gates are pairwise qubit-disjoint.

D. Intra-Core SWAP Scoring

For a local-needing-SWAP gate set F_{loc}, PORTER builds candidate SWAP edges from the coupling-graph neighbours of every physical position touched by F_{loc}, simulates each SWAP, and scores it with SABRE's front-plus-lookahead form [12]:

$$J(a) = \bar{d}_{\mathcal{F}}(a) + \lambda_{\mathrm{loc}}\, \bar{d}_{\mathcal{E}}(a), \qquad \lambda_{\mathrm{loc}} = 0.8, \tag{1}$$

where $\bar{d}_{\mathcal{F}}(a)$ is the mean, over the gates $(q_i, q_j) \in F_{\mathrm{loc}}$, of the intra-core distance $d_{\mathrm{L}}\big(c, \ell_a(q_i), \ell_a(q_j)\big)$ under the post-SWAP layout ℓ_a, and $\bar{d}_{\mathcal{E}}(a)$ is the same mean taken over an extended set $\mathcal{E}_{\mathrm{loc}}$ of up to N_{loc}=20 local CX gates collected from the next L_{loc}=12 DAG layers via a virtual frontier simulation (Section IV-A). Ties are broken first by the raw front-layer distance, then, if still tied, by a decision uniformly sampled from a constant-seeded random-number generator, with every such tie-break recorded in `RouterStats` for reproducibility auditing.

E. Plan-First Remote Stage

PORTER manipulates remote work as *plans*. A plan is a complete, committable specification of one inter-core action: which primitive fires, which gate or same-control run it resolves, its direction, and consequently its *readiness conditions* — which logical endpoint must sit adjacent to which communication port, and which ports must be free. What it deliberately leaves unspecified is the SWAP sequence that establishes those conditions. Alternatives are compared at the level of *what* remote work to do next, each scored as if its preconditions were met; only the committed plan is handed to the machinery that works out *how*.

Both baselines decide at a finer grain. TELESABRE's pool holds staging SWAPs and remote operations together, so the choice is made implicitly, one micro-move at a time: every staging SWAP it selects is also a vote for whichever remote action that SWAP happens to serve, votes for competing actions interleave, and staging effort is routinely spent on actions never taken. DSABRE does separate the stages, but its candidate is a single teleport hop rather than a plan, and its score adds that hop's staging and eviction cost directly to the progress it buys, so a hop cheap to stage now can outrank one that resolves more gates. PORTER ranks by gain alone, demotes readiness cost to a tie-break, and refuses to reopen the choice until the selected plan is ready (Appendix F); Section IV-I measures what that persistence is worth.

When the front holds only remote CX gates, PORTER generates two disjoint candidate sets. **Normal plans** $\mathcal{P}$: for every adjacent-remote CX, a TELEGATE; a TELEGATEPACKET if a same-control run of ≥ 2 remote CX gates can be collected by forward DAG simulation (Section III-G); and, for either endpoint, a TELEDATA plan into the partner's core if that core has at least $f_{\min}$=2 free positions. For non-adjacent-remote CX gates, only distance-reducing TELEDATA hops with sufficient destination capacity. **Blocked plans** $\mathcal{B}$: the same TELEDATA candidates for destinations that currently fail the $f_{\min}$ test. These are never executed; they exist to detect when no normal plan exists because every destination is saturated, or when one is available but a blocked plan promises much more if capacity were relieved first.

Every plan is scored (Section III-F) *before* any preparation SWAP for it exists. PORTER then selects the best plan in $\mathcal{P}$ that is ready or admits an improving preparation SWAP; compares its gain against the best of $\mathcal{B}$ and, if the blocked plan leads by more than m_{assist}=5.0, runs opportunistic capacity relief for that saturated core instead (Section III-H); and, if $\mathcal{P}$ is empty, runs the same breadth-first relief search directly on $\mathcal{B}$'s destinations, shortest hop count first. Once committed, it generates and applies *only that plan's* preparation SWAPs until the plan is ready, no improving SWAP remains, or the full decision state — plan identity, layout, RNG state — repeats, which raises rather than looping.

Preparation SWAPs. A committed plan's readiness requirements are exactly two *(core, neighbour-core, endpoint)* triples (source and destination side for TELEDATA; control and target side for TELEGATE/TELEGATEPACKET, whose target-side routing is deferred to execution time, Section III-G). For each unmet requirement, candidate preparation SWAPs are restricted to edges incident to either the plan's logical endpoint or a nearest free physical position tied for minimum distance to the relevant communication port (only while that port remains occupied); each candidate is scored by the post-SWAP value of the same readiness-cost function used to rank blocked plans (Section III-F), and at most one improving SWAP per core side is applied per pass.

F. Plan Scoring: Direct, Latent, and Lookahead Gain

Every plan scores $\mathrm{total} = \mathrm{direct} + \mathrm{lookahead}$, computed per plan kind but sharing one Δ-form convention: for a CX gate $g = (q_i, q_j)$ and a candidate relocation, $\Delta(g) = d_{\mathrm{C}}(\kappa(q_i), \kappa(q_j)) - d_{\mathrm{C}}(\kappa'(q_i), \kappa'(q_j))$, the reduction in core-graph distance between g's two cores.

A **TELEGATE** contributes a constant direct gain of 1 — one remote CX resolved — and no lookahead. A **TELEGATEPACKET** contributes a direct gain equal to the number of remote CX gates in the collected packet; every gate in an accepted packet is current rather than speculative work, since the whole packet executes under one cat-entangler resource once started. A **TELEDATA** plan decomposes into three tiers, all computed by simulating the DAG frontier under the hypothetical post-teleport core assignment: *direct* — front-layer CX gates already involving the moving qubit, plus the seed gate; *latent-direct* — CX gates on the moving qubit that surface while the simulator can still progress using only 1Q and now-local gates, which are current pressure rather than prediction because the candidate teleport fixes their core distance the moment they appear; and *lookahead* — gates on the moving qubit found in up to L_{tp} further simulated layers, a gate in layer k entering with weight $w_k = \max(w_{\min}, w_0\gamma^{k-1})$, w_0=0.9, γ=0.9, $w_{\min}$=0.5. Thus

$$\mathrm{direct} = \sum_{g \in \mathrm{direct} \cup \mathrm{latent}} \Delta(g), \quad \mathrm{lookahead} = \sum_{(g,k) \in \mathrm{la}} w_k \, \Delta(g), \tag{2}$$

with latent-direct gates entering at full weight alongside direct ones. A capacity-risk term $1/\max(1, f_{\mathrm{dst}}-1)$ is tracked alongside the score for tie-breaking, but never added to it.

This is the point at which PORTER differs most sharply from DSABRE: preparation cost appears nowhere in total. Plans are ranked on gain, and readiness cost enters only as the third tie-break in the selection key, after gain and plan kind. The horizon L_{tp} is counted in *layers*, and the collection admits at most one gate on the moving qubit per layer, because a qubit has at most one zero-predecessor gate at any time. A gate cap N_{tp} therefore cannot bind whenever $N_{\mathrm{tp}} \geq L_{\mathrm{tp}}$, and the horizon is the single parameter that governs how far the estimate looks. No one value of L_{tp} serves every circuit — Section IV-H shows the best value spans 4 to 128 across the benchmark — so PORTER selects it per circuit rather than fixing it.

G. Same-Control Packet Collection

From a seed adjacent-remote CX $g = (q_c, q_t)$ with control core c_c and target core c_t, PORTER simulates the DAG frontier forward for up to 120 layers, greedily absorbing any front CX sharing control q_c whose target is also in c_t (up to 8 such gates) and any front 1Q gate whose qubit is already in c_t, since those execute freely while the cat resource is live. Collection stops at the first unproductive layer or at the size cap. The window counts simulated *layers*, not collected CXs, because same-control CXs are DAG-chained and surface one at a time; in practice it is a cost guard, not an active knob. Because the simulation admits gates from later generations than the bounded current front, a packet may contain gates a naively front-limited scan could not reach. The collected order is replayed exactly at commit time as one `cat_entangler` (consuming the packet's single pair), one `cat_cx` per member — each target routed to its port by ordinary local SWAPs forbidden from touching the two live ports — and a closing `cat_disentangler`.

H. Capacity Relief

Tail-relief BFS. When every remote plan is capacity-blocked, PORTER runs a breadth-first search from each blocked destination core to the nearest *reservoir* core — one with at least $f_{\min}$ free slots — optionally chained beyond one hop (`capacity_relief_chain_enabled`). Every logical qubit currently in the BFS path's penultimate core becomes a tail-relief TELEDATA candidate into the reservoir, scored and selected by the same plan machinery, grouped and tried in ascending hop-count order.

Opportunistic relief. Even when a normal plan is available, a severely blocked plan can still win if relieving its target core's capacity would unlock substantially more gain (Section III-E); this differs from tail-relief BFS in firing preemptively, before any plan is actually starved, whenever the margin m_{assist} is exceeded.

I. Progress and the Failure Model

Deterministic greedy routing on a capacity-constrained machine can stall, and the mechanisms above are best read as one answer to that risk. PORTER layers four defences.

(i) Keep capacity from reaching zero. A core with no free physical qubit is terminal under the port-based action model: it can never vacate an occupied communication port with local SWAPs, so no inter-core action can touch it and its qubits are stranded. PORTER legalises the entry layout so no core starts full, and admits a TELEDATA plan into core c as a *normal* plan only while $\mathrm{free}(c) \geq f_{\min}$=2. Ordinary routing therefore cannot drive a core into the terminal state; only an externally supplied layout can present one, which is why Appendix B's pytket-dqc source is the only place the compiler reports *infeasibility*.

(ii) Detect starvation rather than thrash. Capacity-blocked candidates are still generated, into a separate set $\mathcal{B}$. They are never executed, but an empty plan set alongside a non-empty $\mathcal{B}$ is an explicit signal that the front is starved *by capacity*, and it names the cores responsible.

(iii) Manufacture capacity where it is missing. Tail-relief BFS turns that signal into work, evacuating a qubit from the penultimate core on the path to the nearest reservoir (Section III-H).

(iv) Bound every loop. Plan sides whose preparation score is infinite are never selected; during preparation the compiler hashes its full decision state — committed plan, layout, RNG state — once per round and raises if it repeats; two budgets bound the loops from outside; and when no plan can be prepared and no relief path reaches a reservoir PORTER raises a diagnostic naming the saturated cores.

We state the resulting guarantee precisely, because it is weaker than it may appear. PORTER is *safe* but not *complete*: it never emits an invalid or partial circuit and always terminates, but termination may be by exception, and it has no analogue of DSABRE's termination-with-success proof (Section V). Its completeness is empirical, and Section IV-I measures what buys it: with both relief tiers active none of the twenty-four instances deadlocks in any protocol; with both disabled, four of the six 80-qubit instances do. No instance in this evaluation is left unrouted, but that is a property of the protocol rather than of the router, and the exception is instructive: at a fixed lookahead horizon, Multiplier on the heavy-hex star of Section IV-G livelocks in the intra-core SWAP loop — capacity never reaches zero, no plan is starved, the diagnostic never fires, and it is the iteration budget that ends the run. Routing a second horizon escapes it (Section IV-H), which is a way of not encountering the failure rather than of ruling it out. Supplying a comparable worst-case guarantee is the clearest piece of future work this design leaves open.

J. Computational Complexity

Let N be the gate count, n the logical-qubit count, K the core count and M the per-core physical-qubit count. Distance tables are precomputed once at construction in $O(KM^2 \log M + K^2 \log K)$, matching the two-level Dijkstra decomposition comparable compilers use [11]. Per iteration, classification and draining cost $O(|F|)$; intra-core SWAP scoring evaluates $O(M)$ candidate edges, each at $O(|F_{\text{loc}}| + N_{\text{loc}})$ over one DAG snapshot reused across candidates; remote-plan scoring evaluates $O(n)$ candidates at $O(N_{\text{tp}})$ over the same snapshot. Free-slot queues are $O(1)$ amortised per layout mutation. Summing over N iterations gives $T_{\text{total}} = O(N(Mn + KM^2 \log M))$ excluding precomputation, in line with other core-local distributed compilers [10], [11].

IV. Experimental Evaluation

A. Implementation and Verification

PORTER is implemented in Python and is publicly available together with the verifier and comparison harness. It is fully deterministic and takes no seed: heuristic ties resolve through a fixed chain of deterministic keys and any residual tie through a constant-initialised generator, so for a fixed circuit and hardware the emitted operation stream is byte-identical across runs, every residual tie-break is counted in the emitted statistics, and behaviour preservation across refactorings is certified by hashing the serialised stream. Compile time is kept low by incremental data structures — chiefly the `DagFrontierSimulator` shared by extended-set collection and teleport lookahead, a copy-on-write view over the live DAG that tracks a private overlay of predecessor-count decrements, so a snapshot costs $O(|\text{frontier}|)$ rather than $O(|\text{remaining circuit}|)$ and independent virtual simulations can run from one live state without interfering — together with incremental frontier maintenance, cached distance tables, $O(1)$ free-slot accounting, and apply/restore SWAP scoring in place of layout cloning. These cut wall-clock time by roughly $4\times$ against a straightforward implementation of the same algorithm, with byte-identical output.

Every reported run is certified by an independent replay verifier that treats the compiler as a black box: from the emitted operation stream, the original circuit and the initial layout alone it re-derives gate coverage (each gate exactly once), per-qubit dependency order, coupling-edge legality of every local operation, port rules for every teleport, telegate and packet, and consistency of every reported statistic. The verifier is itself tested by fault injection — ten classes of mutation applied to passing streams, every mutant rejected (Appendix D). All PORTER numbers in this paper are verifier-certified.

B. Setup

Architectures. We use the two grid-of-grids topologies DSABRE [11] evaluates on: architecture A, a 2×2 array of 3×3 cores (36 physical qubits, four links), and architecture H, a 2×3 array of 4×4 cores (96 physical, seven links). Sections IV-F and IV-G add its 5×5-core scalability series and two networks of IBM 27-qubit heavy-hex tiles. All are read from the baselines' own device files, so every compiler sees identical physical qubit ids.

Benchmarks. We use the same MQT-Bench [9] circuit files DSABRE's own evaluation uses, confirmed by exact CX-count match against its published tables: a 25-qubit suite (AE, GHZ, Graphstate, QFT, QNN, Random) on architecture A and a 64-qubit suite (the same six families, plus QPEexact, QAOA and Multiplier) on architecture H, with QNN the deeper 8,126-CX variant. To test at intermediate scale on the same architecture we add an 80-qubit suite of the same nine families. The 80-qubit AE and QNN instances cannot be produced by the stock generator and are reconstructed to match its own construction exactly — in particular QNN is the deep 12,718-CX ZZ-feature-map circuit, not the degenerate 79-CX chain the current generator emits, whose interaction graph is identical to GHZ's. Appendix A details both reconstructions, their byte-level validation, and the CX counts.

Baselines. We compare against TeleSABRE [10] and DSABRE [11], the latter in its default `dSABRE_BurstExt` mode with its shipped configuration — which since its most recent revision means *safe mode*: capacity as a legality condition with a per-core reserve, BFS slack relay, and a guaranteed per-gate recovery transaction (Section V). Its router has changed materially since the version its published tables report, so every DSABRE number here was regenerated from revision `cb5515b` (2026-08-21) rather than quoted, and the harness that produced them is released with PORTER.

Three protocol decisions keep that comparison honest, and all three favour the baseline. We run DSABRE under its own budget convention, letting all three of its recovery budgets derive from the architecture as its own drivers do rather than imposing values of our own — a binding iteration budget would cut its heuristic search short and hand the remainder to its guaranteed drain at a worse EPR count. We do not charge it for pre-routing layout repair: in safe mode `route` brings an externally supplied layout up to its reserve before the clock starts, and those relay teleports land in its EPR counter although they adjust the *starting layout* rather than route anything; PORTER performs the same repair as a free permutation, so we net it out uniformly. And we hand it both families of its own layout candidates rather than picking one (Section IV-C), which can only strengthen it relative to its own published protocol.

TELESABRE's stock reference C binary crashes deterministically on architecture A — in its Hungarian initial-layout pass, not in routing: a qubit is placed only into a core whose remaining capacity exceeds a hard-coded threshold, capping a 9-slot core at six placements, and each unplaced qubit's -1 sentinel is then used as a negative array index. All TELESABRE numbers here use a minimally patched binary that falls back to the least-loaded core; the patch is in our repository. Every compiler invocation is capped at 900 s wall-clock, uniformly across all three compilers, and a call exceeding it is recorded like any other failure to complete.

C. Initial-Layout Protocols

We evaluate under three initial-layout protocols, designed to separate routing quality from initial-layout quality.

Own-protocol comparison. Each router is first evaluated under its own established best-practice initial-layout protocol. For the two baselines this is best-of-three, matching the convention already used to compare TELESABRE and DSABRE to each other [11]; for PORTER it is a single deterministic run:

- **TELESABRE**: the compiled C binary's built-in hungarian initial-layout heuristic, config seeds 0–2, best EPR count of the successful runs.
- **DSABRE**: Qiskit `SabreLayout` on the corners-removed architecture graph (seeds 0–2, with the two least useful corners of *every* core removed — see below) followed by SABRE-style forward → backward → forward routing passes, best EPR count.
- **PORTER**: a *single* routing pass from its single deterministic temporal locality-greedy layout (Section III-A). PORTER takes no seed and performs no layout search, so its column reports one attempt per circuit where each baseline's reports the best of three; every number is exactly reproducible from the circuit file alone.

DSABRE's corner removal reserves low-degree physical qubits as communication slots by excluding them from the SabreLayout coupling map. Its current rule reserves, in every core, the k minimum-degree vertices farthest from all other physical qubits, with k chosen by a fill-adaptive rule (k=1 at 25 and 80 qubits, 2 at 64, 4 at 100 and 200 on our architectures). An earlier version selected four corners *chip-wide*, which on a core-by-core numbering degenerates to one core's corners; the fixed-two-per-core variant we introduced to correct that is essentially the same rule with k pinned to 2. Neither setting dominates — the adaptive rule wins 80q Graphstate and 100q QPEexact, the fixed rule wins 80q AE, 80q QFT, 100q QFT and 25q Random — so rather than choose we give DSABRE both families and take its best: six candidate layouts instead of three, hence up to eighteen routing passes per circuit against PORTER's one. This can only strengthen the baseline relative to its own published protocol, and it is why our DSABRE numbers sit at or below the figures it publishes for the architectures it reports.

Shared-layout comparison. To separate layout quality from routing quality, both DSABRE and PORTER are then run from an identical externally supplied mapping — TELESABRE's own initial layout, and separately `pytket-dqc`'s hypergraph partition (Appendix B) — so the only difference left is the routing loop. **Cross-layout comparison.** Finally each compiler routes the other's own layout, which closes the remaining cell of the design.

D. Own-Protocol Results

TELESABRE's EPR count is teledata plus telegate operations; DSABRE's is teledata only; PORTER's is teleport plus telegate plus telegate-packet operations (one EPR pair per packet, regardless of how many remote CX gates it amortises). All three report in the same one-EPR-pair unit. SWAP counts intra-core SWAPs only.

Against TELESABRE the result is one-sided. A single PORTER run from one deterministic layout lowers geometric-mean EPR by 59.6%, 41.6% and 68.2% at 25, 64 and 80 qubits despite TELESABRE reporting the best of three attempts. Refinement adds to that at 25 and 64 qubits (63.6% and 47.6%) but not at 80, where the single selected pass is already the stronger of the two (68.2% against 63.2%) — the first place in this evaluation where choosing the lookahead horizon (Section IV-H) beats spending four more routing passes. Every converged circuit improves, from -16.0% to -81.0%. PORTER completes all twenty-four instances, as does DSABRE; TELESABRE completes seventeen, failing on 64q Random and on six of the nine 80-qubit circuits including the 23,381-CX Random.

Two details are worth recording. On all eight 64-qubit circuits TELESABRE completes, our harness reproduces the EPR and SWAP counts DSABRE publishes for it [11] exactly, so the two evaluations share a baseline *measurement*, not merely a baseline tool. And the chain-structured GHZ that defeats locally-greedy schedulers, where TELESABRE spends 16 pairs on a 63-CX circuit, costs PORTER and DSABRE 6 each: PORTER's first-appearance visit order puts successive chain neighbours in one core, dissolving the pathology before routing begins.

Against DSABRE the answer depends on the pass budget and we report every point. A single pass takes the geometric mean only at 25 qubits (43.1 against 44.0) and trails at 64 and 80 (182.6 against 144.6; 46.9 against 36.5, both on the

TABLE I

MAIN COMPARISON, OWN PROTOCOL: EACH COMPILER FROM ITS OWN INITIAL LAYOUT. TELESABRE AND DSABRE REPORT THE BEST OF THREE SEEDS; PORTER ROUTES ONE DETERMINISTIC LAYOUT AT EACH OF THREE CANDIDATE LOOKAHEAD HORIZONS (SECTION IV-H), AND PORTER+R SPENDS FIVE ROUTES ON THE BACKWARD-REFINEMENT SCHEDULE OF SECTION III-B INSTEAD — CLOSER TO DSABRE'S NINE TO EIGHTEEN, THOUGH STILL SHORT OF THEM. RICHER SCHEDULES THAT DO REACH THAT BUDGET, AND PASS DSABRE, ARE IN TABLE VI. Δ_{TS} IS THE EPR CHANGE OF WHICHEVER PORTER COLUMN IS STRONGER ON THAT ROW, AGAINST TELESABRE (%); NEGATIVE IS BETTER. THE BOLDED PORTER COLUMN IS THAT STRONGER ONE — REFINEMENT LEADS AT 25 AND 64 QUBITS, HORIZON SELECTION ALONE AT 80. *TELESABRE DOES NOT CONVERGE IN THREE SEEDS; EACH GMEAN ROW COVERS THE CIRCUITS IT COMPLETES. LOCAL SWAP COUNTS, WHICH FOLLOW THE SAME ORDERING THROUGHOUT, ARE TABULATED IN APPENDIX H.

Suite	Circuit	TELESABRE	DSABRE	PORTER	PORTER+R	Δ_{TS}
25q	ae	119	65	65	65	−45.4
25q	ghz	13	4	4	4	−69.2
25q	graphstate	21	6	7	4	−81.0
25q	qft	138	81	74	74	−46.4
25q	qnn	515	143	119	116	−77.5
25q	random	636	404	401	383	−39.8
gmean (6)		106.6	44.0	43.1	**38.8**	**−63.6**
64q	ae	353	165	157	177	−55.5
64q	ghz	16	6	6	6	−62.5
64q	graphstate	62	15	20	16	−74.2
64q	qft	312	179	262	262	−16.0
64q	qnn	889	538	638	537	−39.6
64q	random	*	732	737	737	—
64q	qpeexact	289	187	222	222	−23.2
64q	qaoa	956	555	775	607	−36.5
64q	multiplier	1601	1285	1076	763	−52.3
gmean (8)		284.5	144.6	166.2	**149.2**	**−47.6**
80q	ae	*	332	299	490	—
80q	ghz	26	7	7	7	−73.1
80q	graphstate	87	20	28	24	−72.4
80q	qft	*	315	446	393	—
80q	qnn	*	791	1070	1181	—
80q	random	*	10551	11040	11040	—
80q	qpeexact	784	347	291	526	−62.9
80q	qaoa	*	862	1122	1122	—
80q	multiplier	*	3793	2411	2699	—
gmean (3)		121.0	36.5	**38.5**	44.5	**−68.2**

TELESABRE-matched basis). Refinement leads at 25 and comes within 3.2% at 64; the γ-search of Section IV-J closes the 64-qubit suite outright (172.7 against 173.1 over all nine circuits) and leaves 1.08× at 80. Per circuit PORTER+R wins seven, loses thirteen and ties four; where it wins it wins large — 64q Multiplier is 763 against 1,285, the widest margin here — and where it loses worst the gap is 1.5×, on 80q QPEexact (526 against 347) and 64q QFT (262 against 179). Both of those are losses for *refinement*: on 80q QPEexact the selected single pass takes the instance outright at 291, which is one of the places the two schedules have changed places. This is a comparison against a target that moved: the DSABRE revision measured here lowers its EPR on every one of our instances relative to its published tables, by up to 40% (Section V). Against that earlier version a single PORTER pass led on all three suites; against this one it does not, and we report the current numbers.

E. Shared-Layout Results

The own-protocol comparison mixes two effects: the quality of each compiler's initial layout and the quality of its routing loop. Table II removes the first — both DSABRE and PORTER route, single-shot, from the identical layout TELESABRE built for itself — leaving a pure routing comparison.

TABLE II

SHARED-LAYOUT COMPARISON: DSABRE AND PORTER BOTH ROUTED, SINGLE-SHOT, FROM THE IDENTICAL LAYOUT TELESABRE BUILT FOR ITSELF, SO THE ONLY DIFFERENCE LEFT IS THE ROUTING LOOP. GEOMETRIC-MEAN EPR OVER THE CIRCUITS TELESABRE COMPLETES AND THEREFORE PROVIDES A LAYOUT FOR — ALL SIX AT 25Q, EIGHT OF NINE AT 64Q, THREE OF NINE AT 80Q. Δ_{TS} COLUMNS ARE EACH COMPILER'S CHANGE AGAINST TELESABRE'S OWN-PROTOCOL RESULT. PER-CIRCUIT ROWS ARE IN APPENDIX H.

		DSABRE		PORTER	
Suite	TELESABRE	EPR	Δ_{TS}	+R	Δ_{TS}
25q (6)	106.6	51.0	−52.2	**47.7**	**−55.2**
64q (8)	284.5	189.2	−33.5	**157.2**	**−44.7**
80q (3)	121.0	74.2	−38.7	**55.7**	**−53.9**
all (17)	173.0	100.9	−41.7	**86.0**	**−50.3**

This is the cleanest comparison in the paper. A *single* PORTER pass loses it: DSABRE is better at every scale, −52.2% against −39.6% at 25 qubits and −41.7% against −35.3% pooled (the single-pass column is in Appendix H; Table II reports the refined one). That is a like-for-like statement about routing effort — DSABRE runs three passes over each layout and PORTER one — and once PORTER is given the same schedule the difference reverses: pooled over all seventeen instances PORTER+R reaches 86.0 against DSABRE's 100.9. So the forward–backward–forward schedule, not the plan-committed loop, is what separates the two compilers on fixed layouts; Section IV-I confirms this from the other direction: removing plan commitment moves EPR by only 0.6%, and in PORTER's favour at that.

Pooled fixed-layout grid. A second source of externally supplied layouts — `pytket-dqc`'s hypergraph placements, routed by both compilers on the 25q and 64q suites (Appendix B; Appendix C offers that partition to PORTER as an extra layout candidate instead) — pools with Table II to give forty-one instances on which DSABRE and PORTER routed the *identical* layout. At one pass each, DSABRE wins twenty to sixteen with five ties, pooled geometric mean 134.2 against 139.0. With the refinement schedule that matches DSABRE's pass count that becomes twenty wins to fifteen with six ties, 126.6 against 134.2 — PORTER ahead. Routing quality, with the layout controlled and the pass budget matched, does not separate these two compilers.

Each compiler on the other's layout. Closing the remaining cell — routing each system from its own and from its counterpart's layout (Appendix H) — gives the same reading from the other side. Down a column the compilers are close: on DSABRE's layout, DSABRE reaches 163.9 and PORTER+R 161.1; on PORTER's, 171.2 and 171.4. Across a row the layouts are not: DSABRE's candidate set is the better start for *both*, by 4.3% for its own router and 8.7% for PORTER's. The best cell is PORTER+R on DSABRE's layout.

At the sizes this grid covers, then, the routing stages are worth about the same given the same start, and what separates

TABLE III
SCALE-UP SUITE: QFT AND QPEEXACT AT 100, 200, AND 360 LOGICAL QUBITS, EPR PAIRS. ARCHITECTURES FOLLOW DSABRE'S OWN SCALABILITY PROTOCOL — 5×5 CORES THROUGHOUT, CORE COUNT $6 \to 12 \to 20$ — READ FROM THE BASELINES' OWN DEVICE FILES SO ALL ROUTERS SEE IDENTICAL PHYSICAL QUBIT IDS; OUR DSABRE COLUMN REPRODUCES THE FIGURES IT PUBLISHES FOR EXACTLY THIS SERIES [11] (201/246/766/896/1890/1938) TO WITHIN THE ONE CANDIDATE LAYOUT OUR PROTOCOL ADDS. THE LAST TWO COLUMNS ARE THE FIXED-LAYOUT CONTROL: PORTER ROUTING DSABRE'S OWN SABRELAYOUT CANDIDATES. "t" IS ONE DSABRE ATTEMPT (SECONDS); ITS PROTOCOL RUNS THREE PASSES OVER EACH OF UP TO SIX CANDIDATES. PYTKET−DQC IS OMITTED: ITS HYPERGRAPH PARTITIONER DOES NOT FINISH IN REASONABLE TIME AT THESE WIDTHS.

		architecture			DSABRE		PORTER, own layout		PORTER, DSABRE layout	
Circuit	CX	n	P	K	EPR	t	1 pass	+R	1 pass	+R
QFT	3,420	100	150	6	200	1.7	198	339	240	273
QPEexact	3,579	100	150	6	246	2.0	200	262	200	283
QFT	7,220	200	300	12	765	6.5	579	1128	1031	1018
QPEexact	7,579	200	300	12	896	7.8	528	979	651	946
QFT	13,300	360	500	20	1890	29.9	1156	2822	1870	2546
QPEexact	13,979	360	500	20	1938	38.9	1224	2342	1444	2416
gmean (6)					**705.7**		**507.7**	930.3	665.7	877.9

TABLE IV
ARCHITECTURE-INDEPENDENCE CHECK: THE NINE-CIRCUIT 64-QUBIT SUITE ON FOUR IBM 27-QUBIT HEAVY-HEX CORES (108 PHYSICAL QUBITS), WIRED AS A *ring* (4 LINKS, EVERY CORE DEGREE 2) AND A *star* (3 LINKS, CORE 0 A HUB EVERY ROUTE MUST CROSS). GEOMETRIC-MEAN EPR; THE RIGHT-HAND PAIR IS THE FIXED-LAYOUT CONTROL, PORTER ROUTING DSABRE'S OWN SABRELAYOUT CANDIDATES. PORTER COMPLETES ALL EIGHTEEN INSTANCES, SO EVERY ROW IS OVER THE FULL NINE; PORTER+R, WHICH DOES NOT SELECT A LOOKAHEAD HORIZON, STILL FAILS STAR MULTIPLIER AND IS THEREFORE "—" ON THAT ROW. PER-CIRCUIT ROWS ARE IN APPENDIX H. DSABRE REPRODUCES ITS PUBLISHED NUMBERS FOR THESE TWO ARCHITECTURES ON ALL EIGHTEEN INSTANCES [11].

		own layout		DSABRE layout	
Core graph	DSABRE	PORTER	+R	PORTER	+R
ring (9)	85.2	86.1	82.3	82.9	**81.4**
star (9)	101.1	122.3	—	98.5	**93.5**
star Multiplier	722	455	—	462	**462**

the two end-to-end is that DSABRE searches six candidate layouts where PORTER builds one — a deliberate trade (Section IV-J) that Section IV-J prices. It does not hold at every size: Section IV-F shows the same control leaving 1.24× unrecovered from 150 physical qubits upward.

F. Scaling to 360 Logical Qubits

Every result so far is at 108 physical qubits or fewer. Table III takes the two circuit families that exist at 100, 200 and 360 logical qubits onto DSABRE's own scalability series: 5×5 cores throughout with the core count growing $6 \to 12 \to 20$, so occupancy is roughly constant while the core graph grows. It is a monotone ladder in the quantity that matters: inter-core traffic scales with core count, and DSABRE's own cost rises $200 \to 765 \to 1{,}890$ on QFT across it.

DSABRE takes all six instances under its own protocol. Refinement helps PORTER on two (200q QPEexact $1{,}343 \to 979$, 100q QFT $446 \to 339$) and is inert elsewhere, improving the geometric-mean ratio from 1.45 to 1.32.

The fixed-layout control does not rescue it, and that is the finding. Handed DSABRE's own candidates, PORTER improves to 1.35× and PORTER+R to 1.24× — an improvement, but nowhere near the parity the same control produces at 25 and 64 qubits (Section IV-E) or on either heavy-hex architecture (Section IV-G), where PORTER+R on DSABRE's layout *wins*. On the largest instance DSABRE's layout is actively worse for PORTER than its own (3,266 against 2,822 on 360q QFT), so the mapping is not even the binding constraint there.

This separates two deficits the smaller suites conflate. Up to 108 physical qubits, controlling the layout and matching the pass budget erases PORTER's gap: what remains there is layout, not routing. From 150 upward it does not, and roughly a quarter survives every control we can apply. Something in the plan-committed loop scales worse than DSABRE's per-hop search as the core graph widens — the lookahead horizons of Section III-F are absolute gate counts and do not grow with K, so at twenty cores a plan is scored on a window covering proportionally far less of the circuit than at four. We report this as measured, without isolating the cause. Both a stronger allocator and a K-aware lookahead are therefore open, and PORTER remains 1.8–6.5× faster per attempt across this series, and 4.5× on the largest instance (6.6 s against 29.9 s on 360q QFT), so the budget for either exists.

G. Architecture Independence: Heavy-Hex Cores

Every result so far uses square grid cores wired in a grid. To test how much of PORTER's behaviour depends on that we adopt DSABRE's own architecture-independence experiment [11]: the same nine-circuit 64-qubit suite on four IBM 27-qubit heavy-hex tiles. Two things change at once. The *core* becomes irregular — degrees over $\{1, 2, 3\}$, six degree-1 leaves, diameter 12 against a 4×4 grid's 6 — and the *core graph* is either a ring, where every core has degree 2, or a star, where core 0 is a hub and the others are leaves. PORTER's hardware object is derived from the architecture DSABRE's own builder produces, and a topology assertion checks both compilers see identical qubit ids, intra-core edges and port endpoints before anything is routed; DSABRE routes on its native object, reproducing its published numbers on all eighteen instances. Nothing in either compiler is specialised for a heavy-hex tile.

The core graph matters; the core shape does not. An irregular tile costs PORTER nothing: on the ring PORTER+R *leads* DSABRE (82.3 against 85.2), winning six of nine and tying GHZ — better relative standing than it holds on the grid suite of Table I. Doubling the intra-core diameter and replacing degree-4 grid interiors with degree-1 leaves disturbs neither compiler. The star does: PORTER+R goes from six wins to three and trails by 13.6% on the eight circuits it completes. PORTER itself wins two of nine there. Note the star rows of Table IV are over all nine circuits and so are *not* comparable with an eight-circuit figure: they include Multiplier, the hardest instance on this core graph, which earlier protocols could not route at all.

Star Multiplier is a livelock the horizon escapes. At a fixed L_{tp}=16 this instance does not terminate, and it fails as a local-SWAP livelock rather than the capacity deadlock of Section III-I: retired gates freeze at 3,226 of 13,040 and the EPR count at 538 while intra-core SWAPs accumulate at one per iteration, so the iteration guard rather than the progress argument stops it. It is not a property of the circuit but of the horizon: the same layout and the same router complete it at L_{tp}=128, at 455 pairs against DSABRE's 722. Since horizon selection routes that candidate as a matter of course (Section IV-H), PORTER has no unrouted instance anywhere in this evaluation. PORTER+R, which holds the horizon fixed, still fails it — the one place where the extra passes are worth strictly less than the extra candidate.

The star gap is the layout, not the routing. Given DSABRE's own layout PORTER completes all nine star circuits — including Multiplier at 462 pairs against DSABRE's 722, which its own layout now also routes, slightly better, at 455 — and with the pass budget matched the control reverses the result on *both* core graphs (-4.4% on the ring, -7.5% on the star), the larger margin falling on the topology PORTER loses outright under its own protocol. The cause is visible in the layout: `build_initial_layout` reserves $f_{\min}$ in *every* core, treating cores as interchangeable. They are on a grid and on a ring, where every core has the same degree; they are not on a star, where Multiplier opens with free counts $(2, 2, 13, 27)$ — the hub every route must cross pinned at the floor while a whole core sits empty. Raising the reservation uniformly is not the repair: at $f_{\min}\in\{4, 6\}$ it helps Random, QNN and QFT but hurts QAOA, QPEexact and GHZ, and Multiplier livelocks at every value we tried. What the star asks for is a reservation scaled by a core's position in the core graph, which we have not implemented.

Taken with Section IV-F this bounds the layout reading rather than extending it: up to 108 physical qubits, controlling the layout and matching the pass budget erases PORTER's gap; from 150 upward roughly a quarter survives every control. The two experiments locate a layout deficit below about a hundred physical qubits and a separate, unexplained routing deficit above it.

H. Selecting the Lookahead Horizon

The teleport-lookahead horizon L_{tp} is the one parameter of Section III-F that no single value serves. Sweeping it over $\{4, 8, \ldots, 1024\}$ on every instance, the best value spans the whole range: 80-qubit QNN wants 4, the 25-qubit suite wants 16, and every 100-, 200- and 360-qubit instance wants 128, beyond which EPR saturates. The gap between the shipped 16 and the per-instance best is 17.2% geometric-mean EPR.

The direction is set by how dense the circuit's interaction graph is. Writing ρ for the mean fraction of the other logical qubits each qubit shares a CX with, a long horizon reports the moving qubit as needed in *every* core at once when $\rho \to 1$, so the gain estimate degenerates into noise and short horizons win; when ρ is small the signal is coherent and long horizons win. Across the benchmark, ρ ranks the best horizon at Spearman -0.765. It is not, however, a usable predictor on its own: in the band $0.42 \le \rho \le 0.53$, where most 64-qubit circuits sit, QAOA wants 8 while AE wants 128, and a threshold rule on ρ recovers only 8.0% of the available 17.2%.

PORTER therefore uses ρ to *prune* rather than to choose. A circuit with $\rho \ge 0.80$ is routed at $L_{\mathrm{tp}} \in \{4, 8, 16\}$ and every other circuit at $\{8, 16, 128\}$; the lowest-EPR route is kept. No instance above the threshold ever selected the long horizon, so pruning costs no EPR, and it removes the case where the long horizon fails to converge and consumes the whole attempt — the dominant compile-time term. The circuit's own default horizon is always among the candidates and wins ties, so selection can never return a worse route than routing once.

Across all 48 instances this yields 23 improvements, no regressions, and 17.4% lower total EPR, at $5.1\times$ the cost of a single pass — still below DSABRE's nine-route protocol (Section IV-J). It is the mechanism behind the scale-up result of Section IV-F and behind PORTER completing every heavy-hex instance in Section IV-G.

TABLE V
MECHANISM ABLATION: PORTER EPR WITH ONE DESIGN DECISION DISABLED AT A TIME, ALL FROM THE SAME TEMPORAL LOCALITY-GREEDY LAYOUT. COLUMNS: "NO PK" DISABLES PACKETS (`MAX_PACKET_SIZE`=1), "NO REL." DISABLES BOTH RELIEF TIERS, "HOP-WISE" REPLACES PLAN COMMITMENT WITH PER-ITERATION RE-SCORING OF THE WHOLE FRONT, "TELEDATA" REMOVES THE TELEGATE FAMILY; "PK" AND "TG" ARE THE PACKET AND SINGLE-TELEGATE COUNTS OF THE FULL CONFIGURATION. THE LAST GMEAN ROW PAIRS FULL AGAINST HOP-WISE OVER ALL TWELVE ROWS, WHICH BOTH COMPLETE. †FAILS TO COMPLETE (DEADLOCK OR ITERATION-GUARD TRIP). ‡COMPLETES ONLY WITH THE ADDITIONAL RELIEF TIER DESCRIBED IN THE TEXT, WHICH THE TELEGATE OTHERWISE MAKES UNNECESSARY. THE FIRST GMEAN ROW IS OVER THE FOURTEEN INSTANCES THE FIRST THREE CONFIGURATIONS SHARE; THE SECOND IS THE TELEDATA-ONLY COLUMN PAIRED AGAINST FULL OVER THE SIXTEEN INSTANCES IT COMPLETES.

Suite	Circuit	Full	No pk	No rel.	Hop-wise	TELEDATA	pk	tg
25q	ae	65	71	80	65	68‡	8	0
25q	qft	74	74	140	74	74	0	0
25q	qnn	119	122	212	128	119	0	0
25q	random	401	412	395	399	413‡	43	93
64q	ae	177	211	327	172	198‡	32	0
64q	qft	262	372	382	336	**216**‡	45	1
64q	qnn	864	931	600	785	**694**‡	240	5
64q	random	737	843	789	805	†	77	228
80q	ae	490	688	†	383	**354**‡	264	15
80q	qft	446	435	†	415	**307**‡	209	2
80q	qnn	1517	1571	†	1474	**926**‡	448	30
80q	random	11040	11628	†	11280	†	572	3538
gmean (14)		57.6	61.4	67.0				
gmean (16)		78.3	83.9			**71.7**		
gmean (12)		413.3			411.7			

I. Mechanism Ablation

Table V attributes PORTER's EPR consumption to its mechanisms by disabling each in isolation, holding the initial layout and every other parameter fixed; the per-action counts behind these rows are tabulated in Appendix G (the six omitted rows — GHZ and Graphstate at every scale — form no packets, never trigger relief and issue no telegates, so all configurations coincide there). Two columns go further than a per-primitive ablation: "Hop-wise" replaces PORTER's

unit of commitment with DSABRE's while holding scoring, plan set and relief tiers fixed, and "TELEDATA" removes the TELEGATE family outright, leaving state teleportation as the sole inter-core action.

Packets. Disabling TELEGATEPACKET makes every shared-control remote CX run pay per gate, and the cost tracks the packet count: $+42\%$ EPR on 64q QFT ($262 \rightarrow 372$, 45 packets), $+40\%$ on 80q AE ($490 \rightarrow 688$, 264 packets), $+19\%$ on 64q AE, against a $+7\%$ overall gmean; the lone counter-move (80q QFT, $446 \rightarrow 435$) shows packet formation is a heuristic commitment that can displace a slightly better teleport plan. Read alone, this column is a clean win for the primitive. The next one overturns that reading.

TELEDATA only. Removing the TELEGATE family exposes two things the per-primitive ablations cannot. First, the TELEGATE is what makes PORTER *terminate*, not merely what makes it cheap: it is the only remote action needing no free destination slot, so it is always a legal plan for an adjacent remote CX. Without it a front whose remote CXs are all core-adjacent and whose destinations sit at or below $f_{\min}$ has an empty plan set, while the tail-relief BFS is only ever offered the *non-adjacent* blocked candidates — ten of the twelve rows deadlock on this. Feeding adjacent-blocked candidates into the same BFS (marked ‡, and measurably inert whenever the TELEGATE is available) recovers all but the two Random rows, which livelock — and those two are the most telegate-heavy in the suite (228 and 3,538 single telegates), so the primitive is load-bearing precisely where it is used most.

Second, where TELEDATA-only completes it is *cheaper than the full action set*, increasingly so with scale: $+2\%$ at 25q, mixed at 64q, and -33% gmean at 80q; overall 8.5% below Full on the sixteen instances it completes. The circuits it wins are exactly the packet-heaviest, making the ordering there TELEDATA only $<$ Full $<$ No packets: packets clearly beat the single telegates they degrade into, but on structured circuits at scale a pure teleportation policy beats both. We read this as myopia in the plan comparator rather than a verdict on packets — a packet's benefit is realised immediately and in full, a teleport's is latent and discounted across a bounded horizon — but we report it as measured. It also answers a question DSABRE states as open: its teledata-only vocabulary was chosen by measurement, an ungrouped telegate serving one gate where teledata amortises many, with grouping left as the open case. Grouping does close that per-EPR gap against single telegates — and still does not justify spending pairs that teleporting would have spent better.

Plan commitment. Replacing PORTER's unit of commitment with DSABRE's — returning to the top of the loop after one round of preparation SWAPs, so the front is re-classified and every plan re-scored before the next move — leaves EPR consumption statistically unchanged: 413.3 against 411.7 over the twelve rows here and 148.8 against 148.0 over the twenty-one instances we ran it on, with eight circuits better without commitment, five worse, eight tied, and all twenty-one completing either way. The two also make the *same number of routing moves* (128,222 against 128,565 iterations), which is the key number: commitment does not change what the compiler decides, only what each decision costs. What it buys is compile time — $1.31\times$ in geometric mean and $3.07\times$ on 80q Random — because a committed plan is not re-derived and re-scored every iteration. It is therefore an efficiency mechanism rather than a quality one: what makes a single PORTER pass cheap enough to be a competitive operating point, not what makes that pass good.

TABLE VI

PROTOCOL TRADE-OFF OVER ALL 24 INSTANCES. "ROUTES" IS THE NUMBER OF ROUTING PASSES; TIMES ARE COMPLETE COMPILE COSTS (GEOMETRIC MEAN PER INSTANCE). PORTER ROUTES ONE CANDIDATE LOOKAHEAD HORIZON PER SET OF THREE (SECTION IV-H); PORTER+R ADDS BACKWARD REFINEMENT (SECTION III-B) AT A FIXED HORIZON; PORTER+S ADDS THE BEST-OF-γ LAYOUT SEARCH OF THIS SECTION ON TOP OF IT; PORTER+SR TAKES THE BETTER OF HORIZON SELECTION AND THAT SEARCH, NETTING THE ONE ROUTE THEY SHARE. ALL REMAIN DETERMINISTIC AND SEED-FREE.

Protocol	routes	EPR	vs. DSABRE	time (s)	vs. DSABRE
DSABRE	9–18	163.9	1.00	2.26	1.00
PORTER	3	174.9	1.07	1.32	**0.58**
PORTER+R	5	171.4	1.05	1.67	0.74
PORTER+S	9–13	163.2	1.00	4.59	2.03
PORTER+SR	11–15	**158.2**	**0.97**	5.65	2.50

Capacity relief. Disabling both relief tiers is not merely more expensive — it is *unsafe*: 80q AE, QFT, QNN and Random all fail to complete, because with every candidate destination below $f_{\min}$ and no mechanism to drain qubits toward reservoirs the deterministic-greedy loop has no legal remote plan left. Where routing does complete, dense circuits pay heavily ($+89\%$ on 25q QFT, $+78\%$ on 25q QNN, $+85\%$ on 64q AE). The two instances where relief *hurts* — 64q QNN ($864 \rightarrow 600$ without it) and, marginally, 25q Random ($401 \rightarrow 395$) — show the opportunistic tier can overpay where congestion would have resolved itself. Of the four mechanisms this table ablates, capacity relief is the one that earns its place outright: it converts the raise-on-deadlock failure model of Section III-I from a liability into a non-event. (The initial layout, which this table holds fixed rather than ablating, earns its own place separately — Sections IV-E and IV-G.)

J. Operating Points and Compile Time

PORTER's protocol routes one deterministic layout at each of three candidate lookahead horizons and keeps the best (Section IV-H), and two further schedules spend more routes: PORTER+R adds the backward refinement of Section III-B, and PORTER+S additionally sweeps the one free scalar in the layout heuristic — the per-layer discount γ, pinned at 0.99 elsewhere — over $\{0.90, 0.95, 0.99, 1.0\}$ and keeps the best route by EPR. Both remain deterministic and seed-free, and the sweep is a search inside PORTER's own design rather than an external partitioner or a seed sweep. A fourth point falls out of the first three. Selection and the γ-search look for different things — the horizon at a fixed layout, against the layout at a fixed horizon — and neither dominates: selection gives the better route on ten of the twenty-four instances and the search on fourteen. PORTER+SR runs both and keeps the lower EPR, netting the one route they share. Table VI places the four points against DSABRE, pooled over all twenty-four.

The trade is time against EPR pairs, and it is monotone in both. PORTER costs $1.07\times$ DSABRE's EPR at $0.58\times$ its compile time; refinement reaches $1.05\times$ at $0.74\times$; the layout search reaches parity (163.2 against 163.9) at $2.03\times$; and PORTER+SR passes it, at $0.97\times$ its EPR for $2.50\times$ its compile time. Nothing here is free, because PORTER is dearer per routing pass and the richer schedules run more of them — 12.4 on average for the γ-search alone.

That last point answers the question Table I raises. PORTER+SR leads DSABRE on twelve of the twenty-four instances against nine behind, and by construction never loses to either schedule it is built from. The residual own-protocol gap is therefore a *search-budget* gap, not a routing deficit: given a budget in DSABRE's range PORTER passes it rather than merely drawing level. Where on the curve to sit depends on whether EPR pairs or compile seconds are scarce.

Selecting the horizon has largely overtaken refinement as an *operating point*. Over the twenty-four instances PORTER+R is ahead by only 2.0% in geometric mean, winning nine and losing five, for four extra routes; it is 12.0% *behind* the single selected pass at 80 qubits, 83% behind on the scale-up series of Section IV-F, and it is the configuration that fails star Multiplier in Section IV-G. We nonetheless report PORTER+R throughout, because it is what makes the fixed-layout comparison of Section IV-E a fair one: that experiment holds the initial layout constant and asks about routing alone, so it needs a PORTER configuration whose extra effort goes into routing passes rather than into layouts, and PORTER+S — which searches over layouts — cannot serve that role. PORTER+S earns its place for the opposite reason: it never loses to PORTER+R on any of the twenty-four instances (eleven wins, thirteen ties), and it is half of PORTER+SR, the point that answers whether the own-protocol gap is a routing deficit or a search-budget one. Neither schedule is redundant: they are the two halves of that answer, and each is the only one that can play its own role — PORTER+R because the fixed-layout control must not search over layouts, PORTER+S because selection alone does not reach DSABRE.

All times are *complete* compile costs, following DSABRE's own convention: layout generation plus every routing pass over every candidate, not the winning attempt alone. For DSABRE that is nine to eighteen passes, against three for PORTER and five for PORTER+R; TELESABRE's figure covers all three seeds.

The cost is per pass, and it rose. Per *routing pass* PORTER is now the slower of the two on all nine 64-qubit circuits — geometric mean $0.08\times$ DSABRE's per-pass speed — partly what its remote stage costs (enumerating whole plans, collecting packets by forward DAG simulation and running a two-tier capacity search are all dearer per iteration than scoring a flat list of hops) and partly the horizon: a deep peel makes each iteration substantially more expensive, which is the price of the candidate that wins. Pooled over the whole suite the protocol still comes out ahead — $0.58\times$ DSABRE's complete cost (Table VI), because three PORTER routes remain cheaper than nine DSABRE ones — but on this nine-circuit 64-qubit subset the complete costs are 1.98 s against 1.86 s, and DSABRE is the cheaper of the two. Where PORTER was uniformly faster per unit of work before selection, it is now faster only in aggregate.

Over the five circuits TELESABRE completes (per-circuit times in Appendix H, all four columns except TELESABRE's re-measured in one session), geometric-mean complete cost is 5.95 s for TELESABRE, 2.67 s for DSABRE, 1.55 s for PORTER and 1.97 s with refinement. Neither PORTER point survives scale intact: on the two largest circuits here PORTER+R costs 9.9 s against DSABRE's 7.9 s at 8,126 CX and 191 s against 144 s at 23,381. The cheap operating point is still PORTER, and at $1.07\times$ DSABRE's EPR it no longer gives up much to be so.

V. Related Work

TELESABRE [10] is the published state of the art for this cost model: it supports teledata and gate teleportation and builds on SABRE's extended-set scoring. The two differ chiefly in decision structure. TELESABRE scores inter-core candidates through a contracted-communication-graph Dijkstra energy that folds staging cost and lookahead into one value, and — uniquely among the compilers here — places those candidates in the *same* pool as local SWAPs, taking the global minimum each iteration. PORTER keeps local work strictly ahead of inter-core work, scores whole plans independently of preparation cost, generates preparation SWAPs only for the plan it commits to, and adds the packet primitive. Its capacity handling is also more proactive: alongside TELESABRE's safety-valve cap it maintains an explicit blocked-plan set with a tail-relief search and a pre-emptive relief comparator.

DSABRE [11] is concurrent work on the same problem, and the honest description is that the two compilers are close relatives developed in parallel. Its router was substantially revised while this paper was in preparation — a shared extended-set construction and a safe-mode capacity discipline — lowering its EPR on every one of our own-protocol instances relative to its published tables, by up to 40%, and cutting its compile time by up to $97\times$. We therefore re-ran it from source throughout rather than quoting figures its current code does not reproduce, and our numbers match those it now publishes for the architectures it reports.

Stating what the two share keeps either from being credited with the other's ideas: both rank moves by SABRE's front-plus-lookahead Δ score; both are strictly locality-first, scoring an inter-core candidate only once no front gate is intra-core; both promote capacity from a scoring penalty to a legality condition with the same floor of two free slots; and both manufacture capacity by relaying a free slot from a reservoir along a BFS path rather than evicting arbitrarily. Four differences remain, and they are what this paper evaluates: PORTER's action vocabulary is larger (DSABRE ships teledata only); its unit of commitment is a whole plan rather than one hop; preparation cost enters as a tiebreaker rather than additively; and its initial layout is one deterministic construction rather than a seeded best-of-N search. DSABRE's safe mode additionally carries a termination-with-success proof under stated architectural hypotheses. PORTER is *safe* but not *complete*:

it never emits an invalid circuit and always terminates, but termination may be by exception (Section III-I). Supplying a comparable guarantee is the clearest piece of future work this design leaves open.

Other approaches. Partition-first compilers assign qubits to cores by hypergraph or community partitioning [13]–[15] and route the residue; `pytket-dqc` [14] is the mature representative, compared against in Appendix B under the caveat that its e-bit unit assumes long-lived entanglement and different communication memory. AutoComm [8] identifies gate sequences that can share one entangled resource; PORTER's packet is that amortisation applied inside a routing loop rather than as a pre-pass. SABRE [12] supplies the scoring skeleton all three compilers inherit.

VI. Conclusion

PORTER is a DQC compiler built around a plan-committed remote stage, a two-tier capacity-relief mechanism, a deterministic seed-free initial layout and a per-circuit choice of lookahead horizon. Against TeleSABRE it lowers geometric-mean EPR consumption by 59.6%, 41.6% and 68.2% at 25, 64 and 80 logical qubits and completes every instance in all three suites, including the seven TeleSABRE cannot converge on. Against dSABRE, concurrent work whose current router we re-ran rather than quoted, it is competitive: $1.07\times$ its pooled EPR at $0.58\times$ its compile time, $1.05\times$ at $0.74\times$ with refinement, parity at $2.03\times$ with a γ-search, and ahead on forty-one identical externally supplied layouts, on both heavy-hex core graphs and on a 100–360-qubit series.

Two of the four mechanism answers are negative, and we think they are the more useful half. Capacity relief earns its place outright — without it four of six 80-qubit instances deadlock — and the initial layout earns its below roughly a hundred physical qubits, where controlling it erases the difference between the two compilers. Above that scale $1.24\times$ survives every control, localising a routing-side deficit we do not explain. Plan commitment does not lower EPR consumption: it makes the same number of routing moves as re-scoring the front each iteration and lands within 0.6%, so it is a compile-time optimisation worth 1.3–3×, not a quality mechanism. And the packet primitive is beaten at 80 qubits by removing gate teleportation entirely, by 32.8% — a negative answer to a question dSABRE states as open.

Three openings follow: a stronger allocator below a hundred physical qubits, lookahead horizons that grow with the core count above it (PORTER's are absolute gate counts, our leading and untested hypothesis for the residual scale-up gap), and a comparator that prefers teleportation where its latent value dominates, since the packet result reads as myopia in plan scoring rather than a verdict against amortisation itself.

Acknowledgments

This work was partially supported by the Natural Science Foundation of China (Grant No. 12471437). The author used Anthropic's Claude (via Claude Code) to assist with: (i) reviewing and applying behaviour-preserving performance fixes to the PORTER implementation, (ii) authoring the independent replay verifier and the cross-router comparison harness, (iii) diagnosing and patching the TeleSABRE reference-implementation issue described in Section IV-B, and (iv) drafting the LaTeX source of this manuscript. All AI-generated content was reviewed and verified by the author, who takes full responsibility for the correctness and originality of the work.

References

[1] D. Cuomo, M. Caleffi, and A. S. Cacciapuoti, "Towards a distributed quantum computing ecosystem," *IET Quantum Communication*, vol. 1, no. 1, pp. 3–8, 2020.

[2] P. Escofet, A. Ovide, M. Bandic, L. Prielinger, C. G. Almudever, S. Feld, E. Alarcón, and S. Abadal, "Revisiting the mapping of quantum circuits: Entering the multi-core era," *ACM Transactions on Quantum Computing*, 2024, arXiv:2403.17205.

[3] L. J. Stephenson, D. P. Nadlinger, B. C. Nichol, S. An, P. Drmota, T. G. Ballance, K. Thirumalai, J. F. Goodwin, D. M. Lucas, and C. J. Ballance, "High-rate, high-fidelity entanglement of qubits across an elementary quantum network," *Physical Review Letters*, vol. 124, no. 11, p. 110501, 2020.

[4] S. Daiss, S. Langenfeld, S. Welte, E. Distante, P. Thomas, L. Hartung, O. Morin, and G. Rempe, "A quantum-logic gate between distant quantum-network modules," *Science*, vol. 371, no. 6529, pp. 614–617, 2021.

[5] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, "Teleporting an unknown quantum state via dual classical and Einstein-Podolsky-Rosen channels," *Physical Review Letters*, vol. 70, no. 13, pp. 1895–1899, 1993.

[6] J. Eisert, K. Jacobs, P. Papadopoulos, and M. B. Plenio, "Optimal local implementation of nonlocal quantum gates," *Physical Review A*, vol. 62, no. 5, p. 052317, 2000.

[7] A. Yimsiriwattana and S. J. Lomonaco Jr., "Generalized GHZ states and distributed quantum computing," *arXiv preprint quant-ph/0402148*, 2004.

[8] A. Wu, H. Zhang, G. Li, A. Shabani, Y. Xie, and Y. Ding, "AutoComm: A framework for enabling efficient communication in distributed quantum programs," in *Proceedings of the 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2022, arXiv:2207.11674.

[9] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking software and design automation tools for quantum computing," *Quantum*, vol. 7, p. 1062, 2023.

[10] E. Russo, E. Vinciguerra, M. Palesi, D. Patti, G. Ascia, and V. Catania, "TeleSABRE: Heuristic layout synthesis in multi-core quantum systems with teleport interconnect," in *Proceedings of the IEEE International Conference on Quantum Computing and Engineering (QCE)*, 2025, pp. 749–758, arXiv:2505.08928. Code: https://github.com/Haimrich/telesabre.

[11] S. Li, "dSABRE: A SABRE-style router for multi-core distributed quantum computers," 2026, manuscript, Centre for Quantum Software and Information, University of Technology Sydney. Code: https://github.com/ebony72/dsabre.

[12] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for NISQ-era quantum devices," in *Proceedings of the 24th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 1001–1014.

[13] P. Escofet, M. Bandic, C. G. Almudever, and S. Feld, "HQA: A hungarian-based qubit assignment for multi-core quantum computers," in *Proceedings of the IEEE Quantum Software Conference*, 2023.

[14] P. Andres-Martinez, D. Mills, T. Forrer, and L. Henaut, "CQCL/pytket-dqc," https://github.com/CQCL/pytket-dqc, Jun. 2024.

[15] O. Daei, K. Navi, and M. Zomorodi-Moghadam, "Optimized quantum circuit partitioning," in *International Journal of Theoretical Physics*, vol. 59, 2020, pp. 3804–3820.



Sanjiang Li received his B.Sc. in mathematics from Shaanxi Normal University in 1996 and his Ph.D. in mathematics from Sichuan University in 2001. He is currently a professor at the Centre for Quantum Software and Information at the University of Technology Sydney (UTS). Prior to joining UTS, he worked in the Department of Computer Science and Technology at Tsinghua University from 2001 to 2008. His primary research interests include knowledge representation, artificial intelligence, and quantum circuit compilation.